

Homework 3 Writeup

Natural Language Processing

Zhizhe Zhang (zz5070)

Q2. Properties of Tokenizers

Q2(b). I used the tokenizer `google-bert/bert-base-cased` from Hugging Face. It is not injective because strings with different whitespace, such as `"hello world"` and `"hello/world"`, can map to the same token sequence. It is also not fully invertible because decoding does not always preserve the original spacing and boundary information exactly.

The same tokenizer also does not preserve concatenation. For example, tokenizing `"h"` and `"ello"` separately does not produce the same result as tokenizing `"hello"` as a whole, because subword tokenization depends on surrounding context and longest useful merges.

Q2(c). In general, a non-trivial tokenizer cannot preserve concatenation because useful tokenization decisions usually depend on adjacent context across the boundary between two substrings. If tokenization were always homomorphic with respect to concatenation, then the tokenizer could not adapt merges, whitespace handling, or word-piece segmentation based on neighboring characters, which removes most of the benefit of learned tokenization.

Q3. Zero-shot Addition

Q3(a). Performance deteriorates from 1-digit to 7-digit addition because long addition requires multi-step algorithmic reasoning, correct carry propagation, and exact copying of several output digits. Language models are much better at recognizing common surface patterns than at executing a reliable digit-by-digit procedure over a long sequence.

Q3(b). The configuration parameters control sampling and output length:

- **temperature:** rescales the next-token distribution. Higher temperature makes output more random and usually less stable.
- **max_tokens:** limits how many new tokens the model can generate. Increasing it allows longer answers, but also gives the model more room to drift.
- **top_p:** nucleus sampling threshold. Higher values allow a larger probability mass of candidate tokens and usually increase diversity.
- **top_k:** keeps only the top k candidate tokens at each step. Higher values increase variety; very small values make decoding more deterministic.
- **repetition_penalty:** discourages repeating previously generated tokens. Increasing it can reduce loops or repeated text, but if set too high it can also hurt faithful digit generation.

In this task, larger sampling freedom usually hurts exact arithmetic because the correct answer benefits from deterministic decoding rather than creative variation.

Q3(c). Qwen3-8B generally performs better than Llama-2-7B on this task because it is a newer and stronger instruction-tuned model. The likely reasons are better calibration on structured outputs, stronger instruction following, and more robust token-by-token generation when the task requires exact numeric patterns.

Q3(d). When `max_tokens` is increased from 8 to 20, the model usually becomes less reliable at exact-match evaluation. Without the prior that the output should stop after an 8-digit sum, it has more freedom to generate extra text, repeat the prompt pattern, or continue after the answer, which increases the chance of malformed outputs even if the correct number appears somewhere in the generation.

Q4. In-context Learning

Q4(a). The baseline one-shot prompt usually improves over zero-shot 7-digit addition, but the improvement is limited because the demonstration is still out-of-distribution: it shows a 1-digit example while evaluation uses 7-digit numbers. The example helps the model infer the input-output format, but it still does not give much evidence for multi-step carry propagation on long numbers.

Q4(b). A simple way to modify post-processing is to extract the integer that appears after the answer cue instead of assuming that the entire generation is only the number. For example, we can search the generated text for a span following **Answer:** or for the last integer on the final generated line, and then parse that digit span into an integer.

After relaxing the output-length constraint, performance often drops in exact-match accuracy even when some outputs still contain the correct number somewhere in the text. The model now has more opportunity to add extra explanation, repeat the prompt format, or continue generating unrelated tokens, so the burden shifts from generation control to robust extraction.

Q4(c). Replacing the 1-digit example with the 7-digit example `1234567+1234567=2469134` should generally improve performance because the demonstration is now much closer to the evaluation distribution. It gives the model a stronger prior on the scale of the operands, the likely length of the answer, and the fact that the task is exact arithmetic rather than free-form explanation.

Setting	Res	Acc	MAE	Prompt length
<code>max_tokens = 8</code>	-0.2053	0.2	820,790.0	79
<code>max_tokens = 50</code>	0.0	0.0	819,866.5	79

Compared with the 1-digit example, the 7-digit example clearly helps when `max_tokens = 8`: in my run, accuracy improved from 0.0 to 0.2. However, when `max_tokens = 50`, the model again tends to continue generating instead of stopping cleanly, so accuracy falls back to 0.0. The more in-distribution example helps the model imitate the correct format and answer length, but it still does not make the model reliably execute long addition.

Q4(d). The error can change across repeated runs when stochastic decoding is enabled, because different random choices can lead to different carry mistakes or different malformed completions. If decoding is made close to deterministic, the outputs become much more stable from run to run.

In the saved debug example, the model sometimes continues the **Question/Answer** pattern and produces an output such as `101010090` for `9090909 + 1010101`, even though the correct answer is `10101010`. This illustrates that the model is often pattern-completing rather than faithfully computing the sum.

One reasonable prompt change is to use a more in-distribution 7-digit example and explicitly request only the final answer on the first line. This could help because it gives the model a stronger format

prior and reduces the chance that it keeps generating additional `Question/Answer` text.

In practice, this helps somewhat but not reliably. The 7-digit example improved exact-match performance in Q4(c) when `max_tokens = 8`, but larger generation budgets still caused continuation and numeric mistakes. So prompt changes can reduce some errors, but they do not fully solve the underlying arithmetic weakness of the model.